Software Requirements Specification: Student Engagement Monitoring System

1. System Overview

The system monitors student engagement patterns across multiple data sources, generates predictive risk scores, triggers automated interventions, and tracks outcomes to enable continuous optimization. The system operates in three integrated tiers: automated monitoring, human intervention management, and outcome analytics.

2. Core Functional Requirements

2.1 Data Integration Layer

Requirement 2.1.1: Student Information System Integration The system must connect to the existing Student Information System via read-only API or database views to extract student demographic data, course enrollment records, and academic history. The integration must run on a scheduled basis, pulling fresh data every 24 hours during term time. Student records must include student ID, name, email, phone number, course code, year of study, residential status (commuter vs on-campus), employment status if available, and previous semester grades.

Requirement 2.1.2: Attendance Tracking Integration The system must ingest attendance data from whatever mechanism the university currently uses, whether that is WiFi-based tracking, card swipe systems, mobile app check-ins, or manual register systems. Data must include student ID, module code, session date and time, attendance status (present, absent, late), and session type (lecture, seminar, lab). The system must calculate rolling attendance percentages over 1-week, 2-week, and 4-week windows for each student in each module.

Requirement 2.1.3: Virtual Learning Environment Integration The system must pull VLE analytics from Moodle or Canvas or whatever platform is in use. Required data points include student ID, module code, login timestamp, time spent on platform per session, specific resources accessed (assignment briefs, lecture notes, rubrics, past papers), forum posts created and read, and assignment draft submissions if the platform supports them. The system must track VLE engagement frequency, recency of last login, and specific interaction with upcoming assessment materials.

Requirement 2.1.4: Library System Integration The system should optionally integrate with library management systems to track physical visits, electronic resource access, and checkout patterns. This is a secondary data source for engagement scoring but provides additional signal about student study behavior.

Requirement 2.1.5: Assessment Submission Integration The system must track assignment submission data including student ID, module code, assessment title, due date, submission timestamp, grade received, and feedback date. The system must flag late submissions and calculate submission timing relative to deadline.

Requirement 2.1.6: Academic Support Integration If the university tracks tutoring appointments, office hours visits, or academic support service usage, the system should ingest this data as a positive engagement signal.

2.2 Data Warehouse and Storage

Requirement 2.2.1: Centralized Data Repository All ingested data must be stored in a centralized data warehouse using either MongoDB for flexible schema or PostgreSQL for relational structure. The database

must maintain historical data for at least three academic years to enable pattern analysis and predictive modeling. Data retention must comply with GDPR requirements and university data governance policies.

Requirement 2.2.2: Data Normalization and Cleaning Before analysis, the system must normalize data formats, handle missing values appropriately, deduplicate records, reconcile student IDs across different source systems, and flag data quality issues for manual review.

Requirement 2.2.3: Real-Time Data Updates High-priority data feeds such as attendance and VLE activity must update at minimum once daily. During peak intervention periods (two weeks before major assessment deadlines), updates should occur every 6 hours to enable timely alerts.

2.3 Risk Scoring Engine

Requirement 2.3.1: Multi-Dimensional Composite Scoring The system must implement a Python-based risk scoring algorithm that combines multiple engagement dimensions into a single composite risk score on a 0 to 100 scale where higher scores indicate higher risk. The scoring model must weight the following factors:

Attendance trajectory: Not just current percentage but whether attendance is improving or declining over recent weeks. A student with 70% attendance that was 85% three weeks ago is higher risk than a student with stable 70% attendance.

VLE engagement frequency: Number of logins per week, time spent on platform, and recency of last access. A student who logged in 15 times last week but zero times this week is flagged.

Assessment-specific preparation: For each upcoming assessment, has the student accessed the assignment brief, downloaded required readings, viewed rubrics or past papers, attended relevant lectures.

Prior assessment performance: Recent grades, submission timing (on-time vs late), grade trajectory (improving vs declining).

Study time proxies: Library usage, VLE time-on-task, tutoring appointments as indicators of time investment.

Requirement 2.3.2: Historical Pattern Matching The system must maintain a database of historical engagement patterns and their associated assessment outcomes. For each current student, the system must identify which historical patterns they most closely match and assign risk scores based on historical failure rates for similar patterns. For example, if students with attendance 60 to 75 percent plus VLE logins under 3 per week plus prior grades 55 to 65 percent had a 68 percent failure rate on subsequent assessments historically, current students matching this pattern receive high-risk scores.

Requirement 2.3.3: Assessment-Specific Risk Calculation Risk scores must be calculated per upcoming assessment, not just per student overall. A student may be low-risk in Module A but high-risk in Module B if their engagement patterns differ across modules. Three weeks before each major assessment deadline, the system must generate assessment-specific risk scores based on preparation indicators specific to that assessment.

Requirement 2.3.4: Contextual Adjustments The risk scoring algorithm must account for student context variables including commuter status (commuters have different engagement patterns that do not necessarily indicate higher risk), part-time employment status (working students have legitimate reasons for lower campus presence), previous academic performance (a student with 70 percent attendance but 85 percent grade average may not be at risk), and identified disabilities or accommodations.

Requirement 2.3.5: Dynamic Threshold Calibration Risk score thresholds that trigger alerts must be dynamically calibrated based on observed false positive and false negative rates. If the system generates excessive false alarms (students flagged who do not actually fail), thresholds should adjust upward. If the

system misses students who subsequently fail, thresholds should adjust downward. This calibration must occur quarterly based on outcome data.

2.4 Alert Generation and Prioritization

Requirement 2.4.1: Automated Alert Triggers The system must automatically generate alerts when students cross defined risk thresholds. Alert timing must be tied to upcoming assessment deadlines. Three weeks before a major assessment, the system must run risk calculations and flag students who meet high-risk criteria for that specific assessment. One week before the assessment, the system must re-run calculations and generate escalated alerts for students who remain high-risk or have worsened.

Requirement 2.4.2: Priority Queuing Alerts must be organized into priority queues for academic advisors based on risk score severity, time until assessment deadline, student response history (students who previously ignored outreach get higher priority), and advisor capacity (distribute workload evenly across available advisors). Advisors must see a dashboard showing highest-priority students at the top.

Requirement 2.4.3: Alert Content Specificity Each alert must contain actionable specifics rather than generic statements. For example, instead of saying "Student X has low engagement," the alert should state "Student X has not accessed the assignment brief for Essay due in 12 days and missed all three lectures explaining the required research methodology." This enables advisors to provide targeted support.

Requirement 2.4.4: Student Dashboard Updates When alerts are generated for staff, the student's own dashboard must simultaneously update with self-service information about their engagement patterns, how they compare to successful peers, and specific action steps they can take immediately without waiting for advisor contact.

2.5 Multi-Channel Intervention Management

Requirement 2.5.1: Email Communication The system must integrate with existing university email infrastructure (Microsoft 365 or Google Workspace SMTP) to send initial outreach emails. Emails must be assessment-specific, personalized with student name and specific engagement gaps, include clear action steps (office hours times, tutoring locations, links to missed materials), and track open rates and click-through rates where technically possible.

Requirement 2.5.2: SMS Communication For students who do not respond to email within 48 hours, the system must trigger SMS outreach via GOV.UK Notify (which provides 2,500 free SMS annually for public sector use). SMS messages must be brief (under 160 characters), conversational in tone, include student first name, reference the specific assessment due date, and provide a simple response mechanism (reply YES for help or call this number).

Requirement 2.5.3: Escalation Workflow Management The system must implement a structured escalation workflow. Day 0: Alert generated, student dashboard updated. Day 1: Automated email sent, logged in system. Day 3: If no response, automated SMS sent, logged in system. Day 5: If no response, alert escalated to advisor for phone call, phone attempt logged. Day 7: If no response, alert escalated to personal tutor for in-person contact, flagged as high-priority non-responder. The system must track each touchpoint and automatically advance through escalation stages based on response or non-response.

Requirement 2.5.4: Response Tracking Every communication attempt and student response must be logged with timestamp, communication channel, message content, student response (if any), and follow-up action taken. This creates an audit trail and enables analysis of communication effectiveness.

2.6 Advisor Interface

Requirement 2.6.1: Risk Dashboard Advisors must have access to a web-based dashboard showing all students currently flagged as at-risk, sorted by priority score. The dashboard must display student name, risk score, specific engagement gaps, upcoming assessment deadlines, previous intervention attempts, and response status. Advisors must be able to filter by module, risk level, response status, or demographic factors.

Requirement 2.6.2: Individual Student Profiles Clicking on any student must open a detailed profile view showing complete engagement history across all modules, attendance trends over time visualized as line graphs, VLE activity patterns, assessment performance trajectory, all previous interventions and outcomes, current risk factors specific to each upcoming assessment, and recommended intervention strategies based on student context.

Requirement 2.6.3: Intervention Logging Advisors must be able to log every contact attempt including date and time, communication method used (email, SMS, phone, in-person), whether contact was successful, what was discussed or what support was offered, what the student committed to doing, and when follow-up is scheduled. All intervention data must be structured to enable later analysis.

Requirement 2.6.4: Bulk Actions Advisors must be able to perform bulk actions on multiple students simultaneously such as sending assessment reminders to entire cohorts, marking groups of students as contacted, or referring multiple students to tutoring services with a single action.

Requirement 2.6.5: Mobile Access The advisor interface must be mobile-responsive to enable advisors to log contacts and view student information from phones or tablets when meeting students in-person or making phone calls away from their desks.

2.7 Student-Facing Dashboard

Requirement 2.7.1: Engagement Overview Students must be able to log into a self-service dashboard showing their engagement metrics for each enrolled module including attendance percentage with comparison to module average, VLE activity frequency with comparison to successful students, upcoming assessment deadlines with preparation status indicators, and overall engagement trend (improving, stable, or declining).

Requirement 2.7.2: Predictive Feedback The dashboard must provide predictive feedback such as "Students with 67 percent attendance typically average 58 percent on assessments. Students with 90 percent plus attendance average 72 percent." This helps students understand the concrete relationship between their engagement and likely outcomes.

Requirement 2.7.3: Assessment Preparation Tracking For each upcoming assessment, the dashboard must show a checklist of preparation steps including attended relevant lectures (yes or no), accessed assignment brief (yes or no), downloaded required readings (yes or no), viewed rubric (yes or no), started work if platform has draft submission feature (yes or no). Items not yet completed must be highlighted as action items.

Requirement 2.7.4: Actionable Recommendations The dashboard must provide specific, clickable action steps such as "Download the assignment brief here," "Book a tutoring appointment here (next available: Tuesday 2pm)," "Access recorded lectures you missed here," or "Office hours: Wednesdays 10am to 12pm, Room 401." Every recommendation must be immediately actionable.

Requirement 2.7.5: Progress Visualization Students must be able to see visual representations of their engagement trends over time using line graphs for attendance trajectory, bar charts comparing their activity to module averages, and color-coded indicators (green, yellow, red) for each module's risk status.

2.8 Outcome Tracking and Analytics

Requirement 2.8.1: Intervention Outcome Recording For every student who receives an intervention, the system must track whether they responded to initial contact, whether their engagement improved after intervention (attendance increased by X percent, VLE logins increased by Y), whether their next assessment

performance improved compared to baseline predictions, whether they submitted on-time versus late, what grade they received, and whether they were retained to next semester. This creates a complete intervention-to-outcome dataset.

Requirement 2.8.2: A/B Testing Framework The system must support randomized controlled trials of different intervention approaches. For example, the system must be able to randomly assign half of flagged students to receive email-first contact and half to receive SMS-first contact, then compare response rates and outcome improvements between groups. This enables evidence-based optimization of intervention methods.

Requirement 2.8.3: Population-Specific Analysis The system must enable analysis of intervention effectiveness across different student populations including residential versus commuter students, working versus non-working students, first-generation versus continuing-generation students, and domestic versus international students. This identifies which interventions work for which populations.

Requirement 2.8.4: Predictive Model Validation The system must continuously validate the accuracy of its risk scoring model by comparing predicted outcomes to actual outcomes. For all students flagged as high-risk, what percentage actually failed? For all students not flagged, what percentage failed anyway (missed students)? The system must calculate and report false positive rate, false negative rate, and overall predictive accuracy on a monthly basis.

Requirement 2.8.5: Advisor Effectiveness Analysis The system must track advisor-level outcomes including average response rate to their communications, average engagement improvement among their students, average assessment performance improvement, and retention rate. This identifies which advisors are most effective and enables sharing of best practices.

Requirement 2.8.6: Return on Investment Calculation The system must calculate financial ROI by comparing the number of students retained who would have otherwise withdrawn, multiplied by tuition fee revenue, minus system operating costs. This provides institutional justification for the program.

Requirement 2.8.7: Quarterly Reporting The system must generate quarterly reports showing total students flagged, total interventions attempted, overall response rate, average engagement improvement, assessment performance improvement, retention improvement, and year-over-year trends. Reports must be exportable as PDF for presentation to university leadership.

3. Non-Functional Requirements

3.1 Performance

Requirement 3.1.1: Response Time The advisor dashboard must load within 2 seconds under normal load. Individual student profile pages must load within 1 second. Risk score calculations must complete within 30 minutes of new data becoming available.

Requirement 3.1.2: Scalability The system must support monitoring of at least 5,000 students simultaneously during pilot phase with architecture designed to scale to 15,000 students for full institutional deployment.

Requirement 3.1.3: Concurrent Users The system must support at least 50 concurrent advisor users accessing dashboards, logging interventions, and viewing student profiles without performance degradation.

3.2 Security and Privacy

Requirement 3.2.1: Data Protection Compliance All student data must be stored and processed in compliance with UK GDPR and Data Protection Act 2018. Personal data must be encrypted at rest and in transit. Access must be role-based with advisors only seeing students assigned to them.

Requirement 3.2.2: Audit Logging Every access to student data must be logged including user ID, timestamp, data accessed, and action taken. Audit logs must be retained for at least two years and be accessible for compliance review.

Requirement 3.2.3: Student Consent and Transparency The system must provide clear student-facing documentation explaining what data is collected, how it is used, how it benefits students, and how students can access or correct their data. Students must be able to view exactly what data the system holds about them.

Requirement 3.2.4: Data Minimization The system must collect only data necessary for engagement monitoring and intervention. Data must be anonymized for aggregate reporting and research purposes. Personal data must be deleted or anonymized two years after student graduation or withdrawal in line with retention policies.

3.3 Reliability and Availability

Requirement 3.3.1: System Uptime The system must maintain 99 percent uptime during term time (excluding scheduled maintenance windows). During assessment deadline periods (two weeks before major deadlines), target uptime is 99.5 percent.

Requirement 3.3.2: Data Backup Student data and intervention records must be backed up daily to university-managed backup systems with ability to restore from backup within 4 hours in case of system failure.

Requirement 3.3.3: Failover Handling If automated alert generation fails, the system must alert technical staff immediately and provide manual workaround procedures for advisors to access risk scores and continue interventions.

3.4 Usability

Requirement 3.4.1: Advisor Training The system must be usable by advisors with minimal training (under 2 hours). The interface must be intuitive enough that occasional users can navigate it without referring to documentation.

Requirement 3.4.2: Student Accessibility The student-facing dashboard must meet WCAG 2.1 Level AA accessibility standards to ensure students with disabilities can access and use the system.

Requirement 3.4.3: Mobile Responsiveness Both advisor and student interfaces must be fully functional on mobile devices with screen sizes down to 375px width (iPhone SE).

3.5 Maintainability

Requirement 3.5.1: Code Documentation All custom code must be well-documented with inline comments explaining logic, README files for each component, and architecture diagrams showing system structure and data flows.

Requirement 3.5.2: Configuration Management Risk scoring algorithms, alert thresholds, escalation workflows, and communication templates must be configurable without requiring code changes. Authorized staff must be able to adjust these parameters through an admin interface.

Requirement 3.5.3: Logging and Debugging The system must maintain detailed application logs to enable troubleshooting of issues. Logs must include timestamps, error messages, stack traces for exceptions, and context about what the system was attempting when errors occurred.

4. Technical Architecture

4.1 Technology Stack

Requirement 4.1.1: Backend Services Backend API and data processing services must be implemented in Python for analytics and risk scoring algorithms or Node.js with Express for API endpoints and workflow management. The choice should prioritize developer familiarity and institutional preferences.

Requirement 4.1.2: Database Data warehouse must use either MongoDB for flexible schema supporting varied data sources or PostgreSQL for relational structure with strong ACID guarantees. Recommend PostgreSQL for pilot given structured nature of educational data.

Requirement 4.1.3: Frontend Interface Advisor and student dashboards must be built using React for component-based UI with responsive design. Use modern React with hooks rather than class-based components for maintainability.

Requirement 4.1.4: Communication Services Email must use existing university SMTP infrastructure. SMS must use GOV.UK Notify API which is free for public sector up to 2,500 messages annually. For scale beyond pilot, consider Twilio or similar commercial SMS gateway.

Requirement 4.1.5: Hosting Infrastructure For pilot phase, leverage existing university cloud infrastructure including Azure or AWS institutional allocations or on-premise servers. Use containerization with Docker to ensure portability between environments.

Requirement 4.1.6: Authentication Use university Single Sign-On (SSO) system for authentication to avoid managing separate credentials. Support SAML or OAuth2 depending on institutional SSO implementation.

4.2 Data Integration Approach

Requirement 4.2.1: API-First Integration Where source systems provide APIs, use those for data extraction. Prefer REST APIs with JSON responses. Implement retry logic and error handling for network failures.

Requirement 4.2.2: Database View Integration Where APIs are unavailable, request read-only database views from source system administrators. Extract data using scheduled SQL queries.

Requirement 4.2.3: File-Based Integration As last resort, support CSV or JSON file uploads from source systems. Provide template formats and validation to ensure data quality.

Requirement 4.2.4: ETL Pipeline Implement Extract-Transform-Load pipeline using Python pandas for data manipulation or dedicated ETL tools like Apache Airflow if available institutionally. Pipeline must run on schedule, log all execution, handle errors gracefully, and alert administrators of failures.

4.3 Deployment Strategy

Requirement 4.3.1: Pilot Deployment Initial pilot must deploy to 2 to 3 high-enrollment first-year modules (approximately 200 to 300 students total). Use this to validate technical architecture, test intervention workflows, gather feedback from advisors and students, and measure outcomes before scaling.

Requirement 4.3.2: Staged Rollout After successful pilot, expand to full department (all modules, approximately 1,000 students), then cross-department (3 to 4 departments, approximately 3,000 to 4,000 students), then institution-wide (all students, approximately 14,000). Each stage must demonstrate positive outcomes before proceeding to next stage.

Requirement 4.3.3: Rollback Plan Maintain ability to roll back to previous version if major issues emerge. Keep previous system version deployable for at least one full semester after new version deployment.

5. Implementation Phases

Phase 1: Foundation (Months 1 to 3)

Month 1: Requirements finalization, technical architecture design, development environment setup, source system access negotiation, initial data extraction testing.

Month 2: Data warehouse schema design and implementation, ETL pipeline development, basic risk scoring algorithm implementation, unit testing of core components.

Month 3: Advisor dashboard MVP development, alert generation logic implementation, email integration, initial testing with sample data.

Phase 2: Pilot Deployment (Months 4 to 6)

Month 4: Pilot module selection, advisor training, student communication about system, system deployment to pilot modules, monitoring for technical issues.

Month 5: SMS integration, student dashboard development, intervention workflow refinement based on advisor feedback, outcome data collection.

Month 6: Pilot outcome analysis, identification of system improvements needed, documentation of lessons learned, preparation of expansion plan.

Phase 3: Iteration and Expansion (Months 7 to 12)

Month 7 to 8: System refinements based on pilot findings, risk scoring algorithm improvements, additional features requested by advisors.

Month 9 to 10: Department-wide expansion, training of additional advisors, monitoring of scale-related issues.

Month 11 to 12: Outcome analysis of expanded deployment, ROI calculation, reporting to leadership, planning for institution-wide rollout.

6. Success Criteria

The software implementation is considered successful if it achieves the following measurable outcomes.

Technical Success Criteria: System maintains 99 percent uptime during term time. Risk scores are calculated and alerts generated within 24 hours of data updates. Advisors report system is intuitive and saves time compared to previous manual processes. No data breaches or privacy violations occur.

Educational Success Criteria: At least 60 percent of flagged students respond to initial intervention. Average engagement improves by at least 10 percentage points among students who respond. Assessment pass rates improve by at least 5 percentage points among intervention recipients compared to control group. Retention rates improve by at least 3 percentage points among at-risk students.

Financial Success Criteria: System prevents at least 20 student withdrawals in pilot year (conservative target). Protected revenue exceeds system costs by at least 10 to 1 ratio. Return on investment is positive within first academic year of operation.